I built a Game Boy emulator in F#

Nick Kossolapov



Why?

- Studied aeronautical engineering in university, not CS
- Been working as a software engineer for about 9 years
- I had no idea how computers actually work

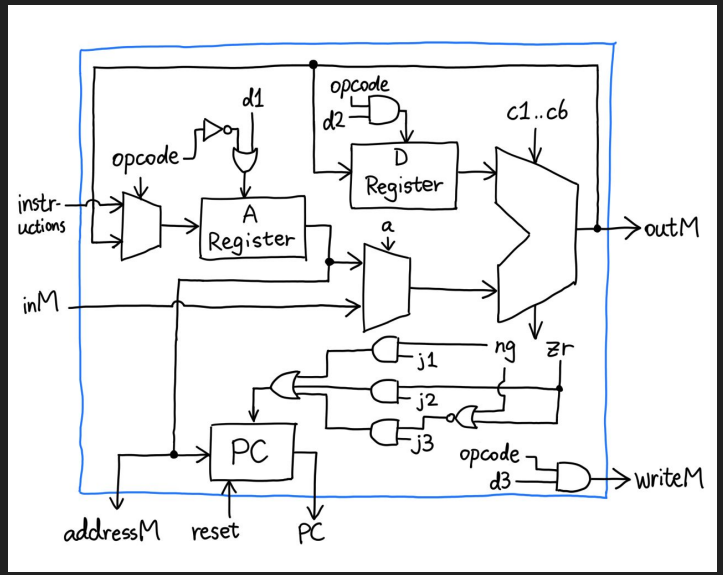
00110110
00111001





From logic gates to an emulator

nand2tetris



CHIP-8



Why F#?

- I like F#
- Rust is too mainstream

What F# brought to Fame Boy

8x	ADD A, B 1 1 Z 0 H CY	ADD A, C 1 1 Z 0 H CY	ADD A, D 1 1 Z 0 H CY	ADD A, E 1 1 Z 0 H CY	ADD A, H 1 1 Z 0 H CY	ADD A, L 1 1 Z 0 H CY	ADD A, (HL) 1 2 Z 0 H CY	ADD A, A 1 1 Z 0 H CY	ADC A, B 1 1 Z 0 H CY	ADC A, C 1 1 Z 0 H CY	ADC A, D 1 1 Z 0 H CY	ADC A, E 1 1 Z 0 H CY	ADC A, H 1 1 Z 0 H CY	ADC A, L 1 1 Z 0 H CY	ADC A, (HL) 1 2 Z 0 H CY	ADC A, A 1 1 Z 0 H CY
9x	SUB B 1 1 Z 1 H CY	SUB C 1 1 Z 1 H CY	SUB D 1 1 Z 1 H CY	SUB E 1 1 Z 1 H CY	SUB H 1 1 Z 1 H CY	SUB L 1 1 Z 1 H CY	SUB (HL) 1 2 Z 1 H CY	SUB A 1 1 Z 1 H CY	SBC A, B 1 1 Z 1 H CY	SBC A, C 1 1 Z 1 H CY	SBC A, D 1 1 Z 1 H CY	SBC A, E 1 1 Z 1 H CY	SBC A, H 1 1 Z 1 H CY	SBC A, L 1 1 Z 1 H CY	SBC A, (HL) 1 2 Z 1 H CY	SBC A, A 1 1 Z 1 H CY
Ax	AND B 1 1 Z 0 1 0	AND C 1 1 Z 0 1 0	AND D 1 1 Z 0 1 0	AND E 1 1 Z 0 1 0	AND H 1 1 Z 0 1 0	AND L 1 1 Z 0 1 0	AND (HL) 1 2 Z 0 1 0	AND A 1 1 Z 0 1 0	XOR B 1 1 Z 0 0 0	XOR C 1 1 Z 0 0 0	XOR D 1 1 Z 0 0 0	XOR E 1 1 Z 0 0 0	XOR H 1 1 Z 0 0 0	XOR L 1 1 Z 0 0 0	XOR (HL) 1 2 Z 0 0 0	XOR A 1 1 Z 0 0 0
Bx	OR B 1 1 Z 0 0 0	OR C 1 1 Z 0 0 0	OR D 1 1 Z 0 0 0	OR E 1 1 Z 0 0 0	OR H 1 1 Z 0 0 0	OR L 1 1 Z 0 0 0	OR (HL) 1 2 Z 0 0 0	OR A 1 1 Z 0 0 0	CP B 1 1 Z 1 H CY	CP C 1 1 Z 1 H CY	CP D 1 1 Z 1 H CY	CP E 1 1 Z 1 H CY	CP H 1 1 Z 1 H CY	CP L 1 1 Z 1 H CY	CP (HL) 1 2 Z 1 H CY	CP A 1 1 Z 1 H CY



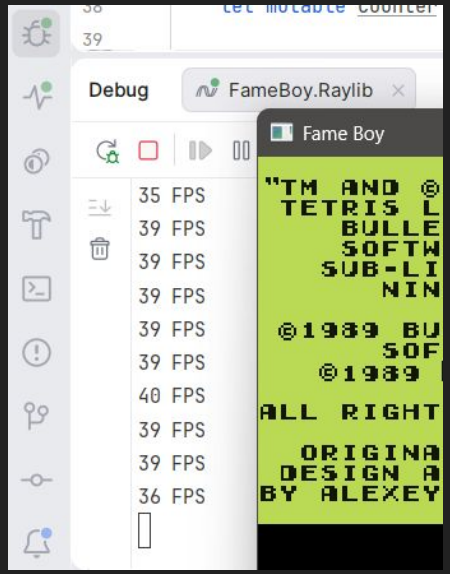
```
type Source =
    | Immediate of uint8
    | RegDirect of Reg8
    | HLIndirect

type Instruction =
    | Add of Source
    | Sub of Source
    | And of Source
    | Or of Source
```

Advantages:

- Compile time safety
- Exhaustive pattern matching
- You feel smart

When you go too far with F#



I tried applying F# types to the memory addresses

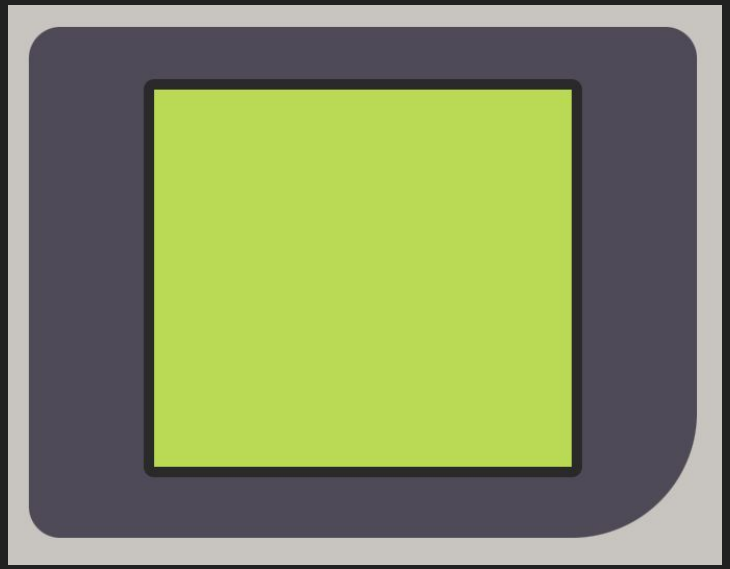
- Used in every instruction, *very* hot path
- Tanked performance
- Creates/destroys millions of objects every sec
- Disrupts branch prediction

But can I make it an Electron app?

F# → Fable → Transpilation → JS



“Tetris”



The Game Boy is an 8-bit system

```
▼ cpu: Cpu
    EnableImeNextInstr: false
    Halted: false
    Ime: true
  ▶ Memory: DmgMemory {romBase: Uint8Arr
    Pc: 532
  ▼ Registers: Registers
      A@: 0
      B@: -15565461
      C@: 16
      D@: 0
      E@: 216
      H@: 93
      L@: 106
      f: 64
    ▶ [[Prototype]]: Object
    Sp: 65534
```

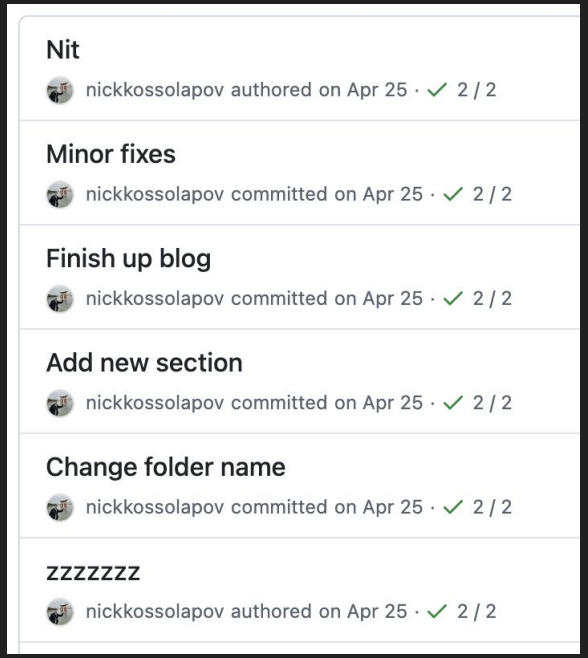


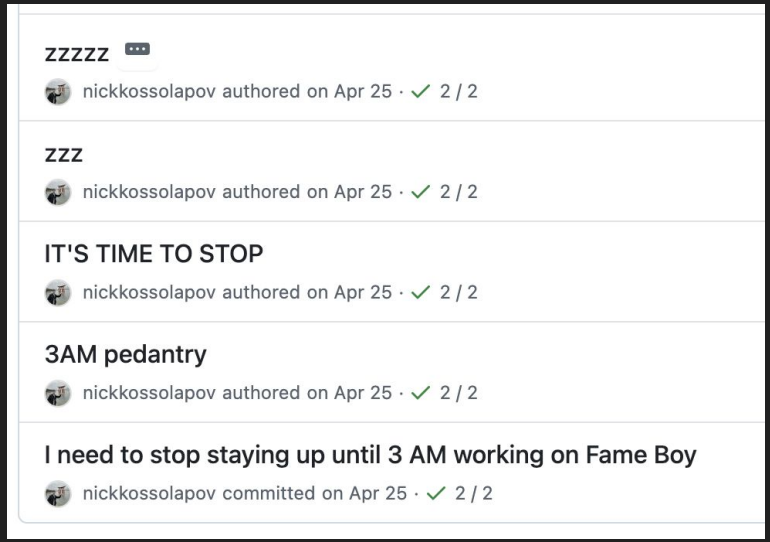
Numbers in .NET vs JavaScript

.NET	JS
sbyte byte short ushort int uint long ulong nint nuint …	Number

What did I get out of it?

Having fun with programming





Blog post with more details

I built a Game Boy emulator in F#

Nick Kossolapov · April 2026

I've been working as a software engineer for over 8 years at this point, and admittedly I've never understood how computers actually work. So I figured I'd try to learn how they work by emulating one. Sorry Ben Eater, I'm not going to build one just yet.

I spent hundreds of hours as a kid catching Pokémon, so the Game Boy was the perfect candidate: real hardware, relatively simple in scope, and something with a strong personal connection.

Instead of jumping straight into it, I first did From NAND to Tetris. It was a great course, and it made me really understand the fundamentals of computers, like registers, memory, and the ALU. Then to get used to building an emulator, I built a CHIP-8 emulator in F#: Fip-8.

A few months later, and after many nights of going to bed at 2 AM even though I told myself I'd only work on it for an hour or two, I have a working Game Boy emulator: Fame Boy. Complete with sound and runs on desktop and web.

Play it in the browser | View on GitHub



Links, blog, slides, and more

https://tinyurl.com/da-game-boy

